TICKET RESOLUTION SYSTEM

Intelligent Multi-Agent Ticket Automation

Powered by RAG · LangGraph · 5 Specialized AI Agents

Multi-Agent AI | RAG Pipeline | LangGraph | FastAPI + React

5
SPECIALIZED AGENTS

RAG
VECTOR KNOWLEDGE BASE

85%
AUTO-CLOSE THRESHOLD

Auto
ZERO MANUAL STEPS

CONTEXT

Problem Statement

Without this system

Engineers manually search past tickets, copy resolutions, update Jira status, and send emails — 10–30 minutes per recurring issue.

With this system

New ticket arrives → agents automatically find similar resolved tickets, email the assignee, and close the ticket if confidence is high enough.

85%
AUTO-CLOSE THRESHOLD

5 min
JIRA POLL INTERVAL

5
AI AGENTS PER TICKET

2
DATA SOURCES

TECHNOLOGIES

Tech Stack

Backend

- FastAPI
- Python 3.11
- LangGraph
- LangChain
- APScheduler
- Qdrant Vector DB
- SQLite DB

Frontend

- React + Vite
- Axios
- CSS Variables
- Dark/Light theme

LLM / Embeddings

- OpenAI GPT-4o-mini
- Google Gemini
- text-embedding-3-small
- Switchable via config

Integrations

- Jira REST API v3
- SharePoint Local
- Gmail SMTP
- CC recipients

Overall Architecture

DATA SOURCES	Jira REST API v3 (tickets + comments) · SharePoint Local
QDRANT VECTOR DB	qdrant/client-a_KB_All.index · L2 score threshold · deduplication upsert
REACT UI SEARCH	POST /api/v1/chat → RAGService → Qdrant→ LLM reranker
APSCHEDULER	Polls Jira every 5 min · triggers agent pipeline on new tickets
5-AGENT LANGGRAPH PIPELINE	Coordinator → Ingestion → Resolution → Notification → Closure
OUTPUTS	Email sent · Jira ticket closed/escalated · SQLite event logged

MULTI-AGENT SYSTEM

5 Specialized Agents

CO

Coordinator Agent

LLM #1

Plans execution order, invokes all sub-agents sequentially, handles failures gracefully, compiles final summary.

IN

Ingestion Agent

LLM #2

Checks KB freshness (1hr threshold). Re-ingests from Jira (+ comments) and SharePoint when stale.

RE

Resolution Agent

LLM #3

Searches Qdrant Vector DB, LLM ranks candidates by relevance. Only passes closed tickets with resolution forward.

NO

Notification Agent

LLM #4

Decides priority (high/normal), builds HTML resolution table, sends email to assignee + CC recipients.

CL

Closure Agent

LLM #5

conf ≥ 0.85 → close in Jira. 0.60–0.85 → escalate for human review. Below 0.60 → skip.

KNOWLEDGE BASE

RAG Pipeline

INGESTION PATH

Jira API + comments → Chunk text → Embed (OpenAI) → Deduplicate by ID → Qdrant upsert

QUERY PATH

User query → Embed query → Qdrant L2 search → Score ≤1.5 → LLM reranker → Results

Deduplication

Re-ingesting a ticket updates its vector — no duplicates. Keyed on Issue Key in stored text.

Comment ingestion

Jira comments (closure reasons) fetched per ticket. Enables resolutions added after closing.

Score threshold

L2 distance ≤ 1.5 (configurable). Filters clearly irrelevant results before LLM reranking.

LLM reranker

LLM evaluates all candidates — keeps only tickets matching the same problem type.

API Endpoints

METHOD	ENDPOINT	DESCRIPTION
POST	/api/v1/chat	RAG search — direct Qdrant, no agent
POST	/api/v1/agent/process-ticket	Manually trigger all 5 agents
GET	/api/v1/agent/status	Check LangGraph availability
POST	/api/v1/tickets/close	Close ticket in Jira + record in DB
GET	/api/v1/tickets/closed/{tenant_id}	Get closed tickets for dropdown
POST	/api/v1/admin/rag-config/ingest/{id}	Run full ingestion pipeline
POST	/api/v1/login	Session-based authentication
GET	/api/v1/health	Server health check

Swagger UI at http://localhost:8000/docs

FRONTEND

UI Features

Chat search

Natural language search — find tickets by describing the problem. LLM reranker ensures only relevant results appear.

Results table

Source, Ticket ID, Description, Resolution, Root Cause, Type, Status, Priority, Confidence. Open tickets show blank resolution.

Close ticket UI

Close button on open rows only. Dialog shows ticket label + dropdown of closed tickets. Resolution auto-fills from selection.

Auto-refresh

After close: triggers re-ingestion → 1.5s wait → re-runs last search. Table refreshes without manual action.

RAG setup page

Configure LLM provider, model, embeddings, Jira credentials. Toggle data sources on/off. One-click ingestion trigger.

Theme toggle

Dark and light modes. Full CSS variable system — all components adapt automatically.

Database Schema

SQLite at backend/database/ticket.db · Single source enforced by app/core/db_path.py

TABLE	PURPOSE	KEY COLUMNS
ticket_events	All lifecycle events	ticket_id, event_type, confidence, resolution, reason
notification_log	Outgoing email log	ticket_id, assignee_email, channel, status, payload
scheduler_processed	Prevents reprocessing	tenant_id, source_type, ticket_id, content_hash
users	Authentication	username, password_hash, created_at
sessions	Session management	session_id, username, expires_at

event_type values: auto_closed escalated notified skipped

Roadmap to Production

PHASE 1 — **POC** — COMPLETE

RAG pipeline, Qdrant vector DB, 5-agent LangGraph system, email with HTML table, React UI with close ticket flow, multi-LLM support (OpenAI / Gemini).

PHASE 2 — **Harden backend** — NEXT

SQLite → PostgreSQL. Redis caching. Proper multi-tenant namespace isolation per client.

PHASE 3 — **Scale agent layer**

Celery + Redis queue replaces APScheduler. Jira webhooks replace polling. LangSmith agent tracing. Retry logic for failed tickets.

PHASE 4 — **Production ready**

Kubernetes, rate limiting, Grafana monitoring, MCP server for Claude Desktop, multi-tenant billing, enterprise SSO.